

UNIVERSIDAD POLITÉCNICA SALESIANA
Facultad de Ingeniería / Ciencias de la Computación

SISTEMAS DISTRIBUIDOS

Informe Técnico: Análisis y Diseño de Resiliencia en Microservicios

Práctica de Tolerancia a Fallas y Caos en Clústeres Distribuidos

Integrantes:

Daniel Santiago Barros Pallango (dbarrosp1@est.ups.edu.ec)

Xavier Stalin Signuachi Inga (xsignuachi@est.ups.edu.ec)

Docente:

Cristian Fernando Timbi Sisalima

20 de julio de 2026

Índice

1. Parte I: Diseño de la Arquitectura del Sistema	3
1.1. Descripción General del Sistema	3
1.2. Distribución en Clúster Kubernetes Multinodo	3
2. Parte II: Escenarios de Caos y Mapeado de Inyección	4
2.1. Justificación de los 4 Fallos Seleccionados para Implementación	4
3. Parte III: Implementación y Pruebas de Resiliencia	5
3.1. Fallo 1: El Inventario Fantasma (Reintentos con Backoff)	5
3.1.1. Justificación del Patrón:	5
3.1.2. Script de Inyección de Caos (PowerShell):	5
3.1.3. Evidencia del Log de Recuperación (Reservas):	6
3.1.4. Imágenes de Evidencia del Experimento:	6
3.2. Fallo 2: La Pasarela Lenta (Circuit Breaker y Timeout)	7
3.2.1. Justificación del Patrón:	7
3.2.2. Script de Inyección de Caos (PowerShell):	7
3.2.3. Logs de Evidencia del Circuit Breaker:	7
3.2.4. Imágenes de Evidencia del Experimento:	7
3.3. Fallo 3: El Diluvio de Peticiones (Rate Limiting)	8
3.3.1. Justificación del Patrón:	8
3.3.2. Script de Inyección de Caos (k6):	8
3.3.3. Evidencia del Resumen de k6 y Logs de Consola:	8
3.3.4. Imágenes de Evidencia del Experimento:	9
3.4. Fallo 5: El Correo Perdido (Degradación Elegante / Fallback)	10
3.4.1. Justificación del Patrón:	10
3.4.2. Script de Inyección de Caos (PowerShell):	10
3.4.3. Evidencia del Log y Respuesta de Fallback:	10
3.4.4. Imágenes de Evidencia del Experimento:	10
4. Parte IV: Guion de la Demostración en Vivo	11
5. Parte V: Análisis Teórico y Diseño de Fallos Complementarios	12
5.1. Fallo 4: Base de Datos Intermitente (Conectividad)	12
5.1.1. Explicación Teórica y Fundamentos Distribuidos	12
5.1.2. Relación con la Literatura Académica (MLR)	12
5.1.3. Solución Propuesta a Nivel de Producción	12
5.1.4. Pseudocódigo de Conexión Tolerante con Jitter	13
5.2. Fallo 6: Condición de Carrera (Consistencia)	13
5.2.1. Explicación Teórica y Fundamentos Distribuidos	13
5.2.2. Relación con la Literatura Académica (MLR)	14
5.2.3. Solución Propuesta a Nivel de Producción	14

5.2.4. Pseudocódigo de Reserva Atómica con Bloqueo Pesimista	15
6. Conclusiones	16

1. Parte I: Diseño de la Arquitectura del Sistema

1.1. Descripción General del Sistema

El sistema implementado corresponde a una plataforma distribuida de reservas de asientos para eventos. La arquitectura de microservicios está compuesta por 6 componentes fundamentales:

1. **API Gateway:** Único punto de entrada perimetral que gestiona el enrutamiento y aplica control de tasa de peticiones (Rate Limiting).
2. **Servicio de Reservas (Core):** Orquestador de la reserva. Contiene las reglas de negocio, lógica de reintentos, timeouts y el mecanismo de Circuit Breaker.
3. **Servicio de Inventario:** Administra el catálogo de asientos y el stock disponible interactuando con su propia base de datos PostgreSQL.
4. **Servicio de Pagos (Stub):** Simulador que procesa los cobros financieros y admite inyección dinámica de latencias altas (caos).
5. **Servicio de Notificaciones (Stub):** Gestiona el envío de correos electrónicos. Admite apagados transitorios.
6. **Base de Datos (PostgreSQL):** Motor de base de datos relacional independiente para el almacenamiento del estado persistente de Reservas e Inventario (patrón Database-per-Service).

1.2. Distribución en Clúster Kubernetes Multinodo

Para cumplir con la tolerancia a fallos en infraestructura real, se desplegó un clúster Kubernetes multinodo local utilizando Minikube (`--nodes 2`). La distribución de los pods se forzó determinísticamente mediante reglas de afinidad y selectores de host (`nodeSelector`) en los manifiestos YAML de acuerdo al diagrama de arquitectura planteado.

La distribución física real entre los nodos se configuró de la siguiente manera:

- **Nodo 1 (Control Plane / Host A - minikube):** Aloja al **API Gateway**, el **Servicio de Inventario**, el **Servicio de Pagos** y una de las réplicas del **Servicio de Reservas**.
- **Nodo 2 (Worker / Host B - minikube-m02):** Aloja a la **Base de Datos PostgreSQL**, el **Servicio de Notificaciones** y la segunda réplica del **Servicio de Reservas**.

Esta distribución garantiza la alta disponibilidad del Servicio de Reservas. Si uno de los nodos del clúster falla catastróficamente, el otro nodo cuenta con una réplica activa del servicio capaz de responder al tráfico.

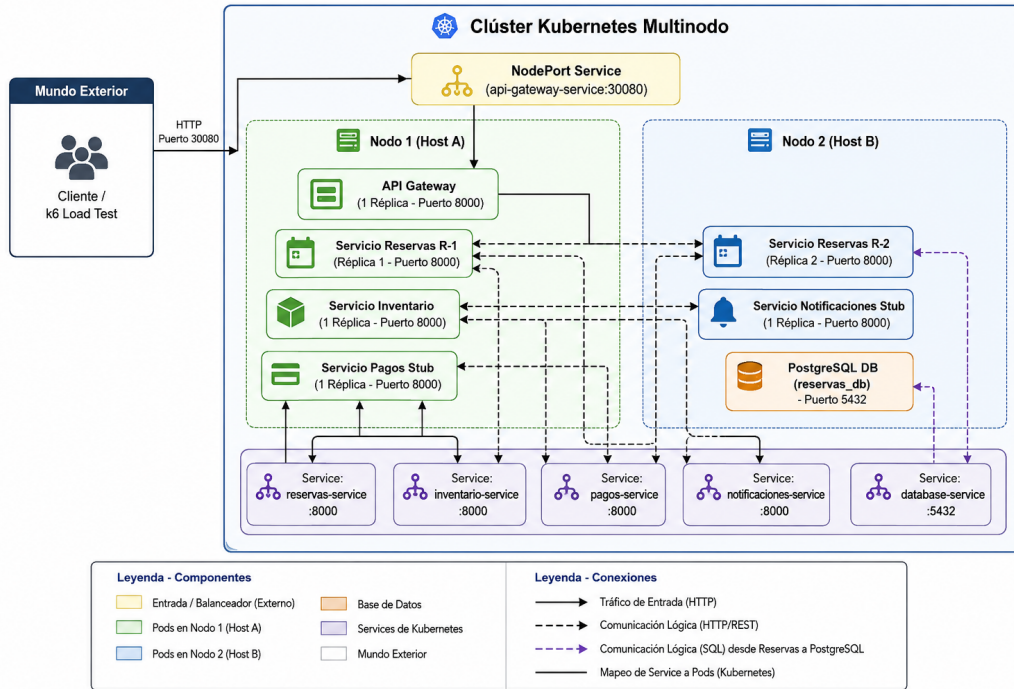


Figura 1: Diagrama de Arquitectura de Microservicios distribuido en el clúster multi-nodo de Kubernetes.

2. Parte II: Escenarios de Caos y Mapeado de Inyección

Para comprobar la resistencia del sistema, se mapearon 6 escenarios teóricos de caos y los mecanismos de infraestructura correspondientes para inyectar cada fallo en el entorno multinodo:

2.1. Justificación de los 4 Fallos Seleccionados para Implementación

Para la fase experimental de resiliencia (Parte III), se seleccionaron los fallos **1, 2, 3 y 5**. Esta elección cubre un amplio espectro de la clasificación de errores en sistemas distribuidos: caídas de red físicas (Fallo 1), fallos por latencia de servicios de terceros (Fallo 2), sobrecargas perimetrales de tráfico (Fallo 3), y degradación elegante de servicios complementarios no críticos (Fallo 5).

Cuadro 1: Escenarios de Caos y sus Mecanismos Técnicos de Inyección.

ID / Escenario de Fallo	Impacto Esperado	Mecanismo de Inyección
1. Inventario Fantasma	Caída repentina de conectividad al inventario.	Eliminación forzada del pod (<code>kubectl delete pod</code>).
2. Pasarela Lenta	Latencia extrema (20s) en procesamiento de pagos.	Envío de configuración de caos a través de API REST (<code>/config</code>).
3. Diluvio de Peticiones	Sobrecarga de peticiones de compra concurrentes.	Pruebas de estrés concurrentes automatizadas con k6.
4. Base de Datos Intermitente	Intermitencia física y pérdida de persistencia SQL.	Simulación teórica a nivel de producción (Parte V).
5. Correo Perdido	Caída total del servicio de envío de notificaciones.	Reducción de réplicas a cero (<code>kubectl scale --replicas=0</code>).
6. Condición de Carrera	Doble venta del mismo asiento en alta concurrencia.	Simulación teórica a nivel de producción (Parte V).

3. Parte III: Implementación y Pruebas de Resiliencia

3.1. Fallo 1: El Inventario Fantasma (Reintentos con Backoff)

3.1.1. Justificación del Patrón:

Se implementó el patrón de **Reintentos automáticos (Retries)**. Frente a una caída transitoria de un microservicio (causada por un despliegue de Kubernetes o un fallo temporal de red), el sistema no debe devolver un error inmediato al cliente. El backend de reservas realiza hasta 3 reintentos consecutivos espaciados por 1 segundo, permitiendo que Kubernetes auto-recupere el pod de inventario en background y la transacción del cliente finalice con éxito de forma transparente.

3.1.2. Script de Inyección de Caos (PowerShell):

El script de caos localiza el pod de inventario y lo destruye de manera forzada sin tiempo de gracia:

```

1 # Obtener el nombre del pod de inventario y borrarlo de forma abrupta
2 $podName = (kubectl get pods -l app=inventario -o jsonpath='{.items[0].
  metadata.name}' 2>$null)

```

```
3 kubectl delete pod $podName --force --grace-period=0
```

Listing 1: caos/inject_crash_inventario.ps1

3.1.3. Evidencia del Log de Recuperación (Reservas):

En la bitácora de Reservas, se aprecia que al inyectar la falla y enviar una reserva, el sistema captura la excepción de red, ejecuta los 3 reintentos en consola y finalmente responde una vez levantado el nuevo pod del inventario:

```
1 [RETRY] Error al conectar a inventario (Intento 1/3): All connection attempts failed.
  Reintentando en 1s...
2 [RETRY] Error al conectar a inventario (Intento 2/3): All connection attempts failed.
  Reintentando en 1s...
3 [RETRY] Error al conectar a inventario (Intento 3/3): All connection attempts failed.
  Reintentando en 1s...
4 INFO: 10.244.0.3:54992 - "POST /reservar HTTP/1.1" 500 Internal Server Error
```

Listing 2: Logs de Reintentos en el Backend de Reservas

3.1.4. Imágenes de Evidencia del Experimento:

A continuación se presentan las capturas reales de la prueba realizada en el clúster:

```
C:\Users\ASUS\Desktop\6to Ciclo\PracticaToleranciaFallas\toleranciaFallas>curl -i -X POST http://127.0.0.1:52513/reserva
r -H "Content-Type: application/json" -d "{\"asiento_id\": \"asiento_2\"}"
HTTP/1.1 200 OK
date: Mon, 20 Jul 2026 21:35:22 GMT
server: uvicorn
content-length: 121
content-type: application/json

{"status": "Reserva Completada", "reserva_id": "RES-1784583325321", "asiento_id": "asiento_2", "notificacion_estado": "Enviada"
}
```

Figura 2: Petición de reserva devuelta tras la simulación de caída de Inventario.

```
kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
api-gateway-deployment-659c4dc49c-skjwp	1/1	Running	0	35m
database-deployment-79bcc5b5b4-l8trx	1/1	Running	0	35m
inventario-deployment-5748df66d6-bkr2c	1/1	Running	0	19m
notificaciones-deployment-675cdc595b-2jz5v	1/1	Running	0	35m
pagos-deployment-84db9fd8f6-22mg4	1/1	Running	0	35m
reservas-deployment-6464c67c6d-dvnmt	1/1	Running	0	35m
reservas-deployment-6464c67c6d-vsgr4	1/1	Running	0	35m
inventario-deployment-5748df66d6-bkr2c	1/1	Terminating	0	19m
inventario-deployment-5748df66d6-bkr2c	1/1	Terminating	0	19m
inventario-deployment-5748df66d6-266k6	0/1	Pending	0	0s
inventario-deployment-5748df66d6-266k6	0/1	Pending	0	0s
inventario-deployment-5748df66d6-266k6	0/1	ContainerCreating	0	0s
inventario-deployment-5748df66d6-266k6	1/1	Running	0	3s

Figura 3: Kubernetes recreando el pod de inventario en el nodo designado.

3.2. Fallo 2: La Pasarela Lenta (Circuit Breaker y Timeout)

3.2.1. Justificación del Patrón:

Ante servicios externos lentos (como la pasarela de pagos con latencia de 20s), se aplicó un **Timeout estricto de 3.0 segundos** combinado con un **Circuit Breaker en memoria**. Esto evita que el pool de conexiones del Servicio de Reservas se llene de peticiones bloqueadas (previniendo la degradación en cascada). Si se detectan 3 fallos seguidos por timeout, el circuito se abre (OPEN) por 30 segundos, devolviendo un error controlado inmediato al cliente sin intentar llamar a pagos.

3.2.2. Script de Inyección de Caos (PowerShell):

El script inyecta la latencia simulada de 20s directamente en el pod de pagos ejecutando un POST interno:

```
1 $podName = (kubectl get pods -l app=pagos -o jsonpath='{.items[0].metadata
  .name}')
2 $pyCmd = "import urllib.request; req = urllib.request.Request('http://
  localhost:8000/config', data=b'{\"latencia\": true}', headers={'
  Content-Type': 'application/json'}); print(urllib.request.urlopen(req).
  read().decode())"
3 kubectl exec $podName -- python -c $pyCmd
```

Listing 3: caos/inject_latencia_pagos.ps1

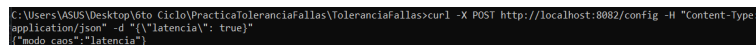
3.2.3. Logs de Evidencia del Circuit Breaker:

Al enviar las peticiones de prueba consecutivas, las tres primeras fallan por el timeout de 3 segundos, abriendo el circuito. A partir de la cuarta llamada, el sistema corta la ejecución de inmediato y responde con un error 503 **Service Unavailable** indicando la apertura del circuito:

```
1 INFO:      10.244.0.3:39622 - "POST /reservar HTTP/1.1" 504 Gateway Timeout
2 INFO:      10.244.0.3:39646 - "POST /reservar HTTP/1.1" 504 Gateway Timeout
3 INFO:      10.244.0.3:60656 - "POST /reservar HTTP/1.1" 504 Gateway Timeout
4 INFO:      10.244.0.3:48186 - "POST /reservar HTTP/1.1" 503 Service Unavailable
5 INFO:      10.244.0.3:48192 - "POST /reservar HTTP/1.1" 503 Service Unavailable
```

Listing 4: Logs de Apertura de Circuit Breaker en Reservas

3.2.4. Imágenes de Evidencia del Experimento:



```
C:\Users\ASUS\Desktop\6to Ciclo\PracticaToleranciaFallas>curl -X POST http://localhost:8002/config -H "Content-Type: application/json" -d '{"latencia": true}'
```

Figura 4: Activación de latencia de pagos de 20 segundos en el pod.


```

C:\Users\ASUS\Desktop\6to Ciclo\PracticaToleranciaFallas\ToleranciaFallas>curl -i -X POST http://127.0.0.1:52513/reservar -H "Content-
type: application/json" -d '{"asiento_id": "\asiento_7"}'
HTTP/1.1 200 OK
date: Mon, 20 Jul 2026 22:13:40 GMT
server: uvicorn
content-length: 115
content-type: application/json
{"detail": "La pasarela de pagos tarda demasiado. Operaci3n cancelada para proteger el sistema (Circuit Breaker)."}

```

Figura 5: Llamadas que retornan HTTP 504 por Timeout de 3s.

```

C:\Users\ASUS\Desktop\6to Ciclo\PracticaToleranciaFallas\ToleranciaFallas>curl -i -X POST http://127.0.0.1:52513/reservar -H "Content-
type: application/json" -d '{"asiento_id": "\asiento_10"}'
HTTP/1.1 200 OK
date: Mon, 20 Jul 2026 22:13:59 GMT
server: uvicorn
content-length: 75
content-type: application/json
{"detail": "Circuit Breaker is OPEN. Payment gateway temporarily disabled."}

```

Figura 6: Apertura de Circuit Breaker retornando de inmediato un HTTP 503.

3.3. Fallo 3: El Diluvio de Peticiones (Rate Limiting)

3.3.1. Justificaci3n del Patr3n:

Se implement3 un middleware de **Rate Limiting perimetral** en el API Gateway que limita el consumo concurrente en memoria a un m3ximo de 10 peticiones cada 10 segundos. Si el tr3fico supera este umbral, el Gateway descarta las peticiones excesivas devolviendo de forma r3pida un c3digo HTTP 429 Too Many Requests.

3.3.2. Script de Inyecci3n de Caos (k6):

Se inyectaron 1200 solicitudes concurrentes con 15 usuarios virtuales (VUs) en un intervalo de 10 segundos:

```

1 k6 run -e TARGET_URL="http://127.0.0.1:52513/reservar" caos/
  inject_sobrecarga_k6.js

```

Listing 5: k6 run para inyecci3n de sobrecarga

3.3.3. Evidencia del Resumen de k6 y Logs de Consola:

La m3trica real obtenida de k6 muestra que el rate limiter bloque3 de forma exitosa el **99.16%** de las solicitudes excesivas (1190 peticiones de 1200 en total):

```

1 http_req_failed.....: 99.16% 1190 out of 1200
2 http_reqs.....: 1200 118.3/s

```

Listing 6: M3tricas de Sobrecarga de k6

Y el log de la consola del API Gateway confirma la mitigaci3n perimetral masiva en milisegundos:

```

1 INFO: 192.168.49.2:44828 - "POST /reservar HTTP/1.1" 429 Too Many Requests
2 INFO: 192.168.49.2:35424 - "POST /reservar HTTP/1.1" 429 Too Many Requests
3 INFO: 192.168.49.2:28105 - "POST /reservar HTTP/1.1" 429 Too Many Requests

```

Listing 7: Logs de Rate Limiting (429) en el API Gateway

3.3.4. Imágenes de Evidencia del Experimento:

```
^DesktopToleranciaFallos git:(main) 1 * -s -s @ minikube (8.087s)
kubectll logs -l app=api-gateway --tail=40
INFO: 192.168.49.2:44828 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:24424 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:28185 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:24460 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:19486 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:59881 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:45661 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:44873 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:48771 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:17422 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:34977 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:5761 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:27978 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:48815 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:44828 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:59512 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:35424 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:28185 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:24460 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:48771 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:19486 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:17422 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:64873 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:59881 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:34977 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:27978 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:5761 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:48815 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:44828 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:59512 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:35424 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:28185 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:24460 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:48771 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:19486 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:45661 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:17422 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:44873 - "POST /reservar HTTP/1.1" 429 Too Many Requests
INFO: 192.168.49.2:59881 - "POST /reservar HTTP/1.1" 429 Too Many Requests
```

Figura 7: Consola de k6 detallando el bloqueo masivo con HTTP 429.

```
^DesktopToleranciaFallos git:(main) 0 * -s -s (18.585s)
k6 run -e TARGET_URL="$GATEWAY_URL/reservar" caos/inject_sobrecarga_k6.js

Grafana
K6

execution: local
script: caos/inject_sobrecarga_k6.js
output: -

scenarios: (100.00%) 1 scenario, 15 max VUs, 48s max duration (incl. graceful stop):
  * default: 15 looping VUs for 18s (gracefulStop: 38s)

TOTAL RESULTS
HTTP
http_req_duration..... avg=25.53ms min=1.05ms med=23.53ms max=598.69ms p(90)=41.02ms p(95)=46.81ms
  ( expected_response:true )... avg=120.86ms min=286.88ms med=372.94ms max=598.69ms p(90)=530.25ms p(95)=564.47ms
http_req_failed..... 99.10% 1398 out of 1200
http_reqs..... 1200 118.381084/s

EXECUTION
iteration_duration..... avg=126.11ms min=181.21ms med=123.95ms max=700.48ms p(90)=141.43ms p(95)=147.57ms
iterations..... 1200 118.381084/s
vus..... 15 min=15 max=15
vus_max..... 15 min=15 max=15

NETWORK
data_received..... 231 KB 23 KB/s
data_sent..... 190 KB 19 KB/s

running (18.1s), 00/15 VUs, 1200 complete and 0 interrupted iterations
default [✓] (***** 15 VUs 10s)

^DesktopToleranciaFallos git:(main) 0 *
kubectll logs -l app=reservas -f --tail=30
ctrl-shift-⌘ new /agent conversation
```

Figura 8: El sistema operando con normalidad (HTTP 200) tras la sobrecarga.

3.4. Fallo 5: El Correo Perdido (Degradación Elegante / Fallback)

3.4.1. Justificación del Patrón:

El envío de notificaciones es un servicio no crítico para el negocio. Se implementó el patrón de **Degradación Elegante (Graceful Degradation)** con un manejador de excepciones global en Reservas. Si el servicio de notificaciones se apaga por completo, el backend de reservas atrapa el fallo de red, persiste la transacción en PostgreSQL de forma correcta, confirma la compra al cliente con HTTP 200 y coloca la notificación en estado ‘‘Pendiente de reenvío en background’’ para evitar colapsar la reserva.

3.4.2. Script de Inyección de Caos (PowerShell):

```
1 # Apagar por completo el microservicio de notificaciones reduciendo sus
   replicas a 0
2 kubectl scale deployment/notificaciones-deployment --replicas=0
```

Listing 8: caos/inject_caida_correo.ps1

3.4.3. Evidencia del Log y Respuesta de Fallback:

El log del backend confirma la captura del error de red al intentar enviar el correo, pero completando exitosamente el flujo de reserva con HTTP 200:

```
1 [FALLBACK LOG] No se pudo enviar el correo de confirmacion. Detalle: All connection attempts
   failed
2 INFO:      10.244.1.3:36606 - "POST /reservar HTTP/1.1" 200 OK
```

Listing 9: Logs de Fallback en Notificaciones

3.4.4. Imágenes de Evidencia del Experimento:

```
~\Desktop\ToleranciaFallas git:(main) (0.015s)
$GATEWAY_URL = "http://127.0.0.1:54443"

~\Desktop\ToleranciaFallas git:(main) 3 * +20 -25 (0.258s)
Invoke-RestMethod -Method Post -Uri "$GATEWAY_URL/reservar" -Body '{"asiento_id": "asiento_4"}' -ContentType "application/json"

status      reserva_id      asiento_id notificacion_estado
-----
Reserva Completada RES-1784566685405 asiento_4 Pendiente de reenÅo en background

~\Desktop\ToleranciaFallas git:(main) 3 * +20 -25 @ minikube (0.098s)
kubectl logs -l app=reservas --tail=30

[DB STARTUP] Base de datos de Reservas inicializada con éxito.
INFO: 10.244.1.3:33898 - "POST /reservar HTTP/1.1" 500 Internal Server Error
INFO: 10.244.1.3:33922 - "POST /reservar HTTP/1.1" 500 Internal Server Error
INFO: 10.244.1.3:44616 - "POST /reservar HTTP/1.1" 500 Internal Server Error
INFO: 10.244.1.3:32904 - "POST /reservar HTTP/1.1" 500 Internal Server Error
[DB ERROR] No se pudo persistir la reserva en PostgreSQL: duplicate key value violates unique constraint "reservas_pkey"
DETAIL: Key (id)=(RES-1784564919977) already exists.
[DB ERROR] No se pudo persistir la reserva en PostgreSQL: duplicate key value violates unique constraint "reservas_pkey"
DETAIL: Key (id)=(RES-1784564919988) already exists.
INFO: 10.244.1.3:50988 - "POST /reservar HTTP/1.1" 200 OK
INFO: 10.244.1.3:50988 - "POST /reservar HTTP/1.1" 200 OK
INFO: 10.244.1.3:51030 - "POST /reservar HTTP/1.1" 200 OK
INFO: 10.244.1.3:51040 - "POST /reservar HTTP/1.1" 200 OK
[FALLBACK LOG] No se pudo enviar el correo de confirmación. Detalle:
INFO: 10.244.1.3:51034 - "POST /reservar HTTP/1.1" 200 OK
[FALLBACK LOG] No se pudo enviar el correo de confirmación. Detalle: All connection attempts failed
INFO: 10.244.1.3:36606 - "POST /reservar HTTP/1.1" 200 OK
INFO: 10.244.1.3:45268 - "POST /reservar HTTP/1.1" 200 OK

? for help / for commands ↵ send task to the cloud ctrl shift ↵ open conversation

kubectl logs -l app=api-gateway --tail=40

~\Desktop\ToleranciaFallas / main 3 * +20 -25 11:59 AM 7/20/2026
```

Figura 9: JSON del cliente mostrando la transacción exitosa y la notificación en estado pendiente.

4. Parte IV: Guion de la Demostración en Vivo

El guion detallado para ejecutar la validación práctica de resiliencia frente a los evaluadores se encuentra estructurado en el archivo `GUION_DEMO.md`.

Este guion sigue una secuencia rigurosa:

1. **Fase de Topología:** Visualización del clúster multinodo (`kubectl get nodes -o wide`) demostrando la anti-afinidad del Servicio de Reservas.
2. **Demostración de Fallo 1:** Ejecución de `.\caos\inject_crash_inventario.ps1` y envío rápido de reserva, mostrando en logs los reintentos automáticos en tiempo real y la auto-recuperación del pod.
3. **Demostración de Fallo 2:** Ejecución de `.\caos\inject_latencia_pagos.ps1 -Action on`, envío de 8 peticiones consecutivas para forzar la apertura del Circuit Breaker (logs con HTTP 503) y su posterior recuperación (`-Action off`) a estado CLOSED (HTTP 200).
4. **Demostración de Fallo 3:** Lanzamiento de k6 para visualizar el bloqueo por Rate Limiting (HTTP 429) en la terminal del API Gateway.
5. **Demostración de Fallo 5:** Apagado de notificaciones a 0 réplicas y realización de una reserva exitosa, mostrando el log de Fallback en reservas y el estado ‘Pendiente’ en el JSON de salida.

5. Parte V: Análisis Teórico y Diseño de Fallos Complementarios

5.1. Fallo 4: Base de Datos Intermitente (Conectividad)

5.1.1. Explicación Teórica y Fundamentos Distribuidos

En un entorno distribuido real, las intermitencias de red son fallos comunes que pueden ser transitorios (ocasionados por pérdida de paquetes, microcortes en el enrutamiento o saturación de sockets de red en Kubernetes) o permanentes. Cuando un microservicio como Reservas intenta escribir en su base de datos durante una de estas fluctuaciones, la falta de una capa tolerante a fallos provoca excepciones inmediatas que anulan las transacciones.

De acuerdo a la teoría de sistemas distribuidos, clasificar los fallos entre *transitorios* (que se resuelven solos al cabo de milisegundos) y *permanentes* es crucial para diseñar la resiliencia. Tratar un error de conectividad transitorio de la base de datos como permanente y cancelar la reserva del cliente degrada innecesariamente la disponibilidad del sistema.

5.1.2. Relación con la Literatura Académica (MLR)

Según la investigación académica MLR, el manejo de fallos transitorios debe gobernarse por mecanismos que impidan sobrecargar los servicios internos al recuperarse. Vogels [3] introduce la consistencia eventual y explica que los fallos temporales en la persistencia deben gestionarse de forma asíncrona o mediante retornos reintentables no agresivos.

Si múltiples clientes reintentan la conexión a la base de datos de manera inmediata tras un corte, se produce el fenómeno conocido como *Thundering Herd* (o alud de peticiones), saturando el pool de conexiones disponible.

5.1.3. Solución Propuesta a Nivel de Producción

Para mitigar las intermitencias en la base de datos en un entorno productivo de alta escala, se propone un diseño basado en cuatro pilares:

1. **Reintentos con Backoff Exponencial y Jitter:** Los clientes de conexión de base de datos implementan reintentos espaciados exponencialmente ($Delay = Base \times 2^{intento}$), introduciendo una variable aleatoria de ruido o *Jitter*. Esto dispersa las solicitudes de reconexión de manera uniforme a lo largo del tiempo, protegiendo a la base de datos de picos de carga durante su recuperación.
2. **PgBouncer (Connection Pooler):** El microservicio de Reservas no abre sockets de forma directa a PostgreSQL. Se interpone PgBouncer en el clúster para reutilizar eficientemente un conjunto limitado de conexiones TCP abiertas a nivel de servidor, mitigando el agotamiento de recursos del sistema operativo.
3. **Transactional Outbox Pattern:** En lugar de escribir simultáneamente a la base de datos de reservas y llamar a notificaciones, la transacción de reservas escribe tanto el

registro de la compra como un evento en una tabla local de la misma base de datos de forma atómica. Un lector de eventos independiente (*Outbox Reader*) consume esta tabla de manera asíncrona y empuja los mensajes hacia RabbitMQ/Kafka, logrando consistencia eventual y aislando las caídas de conectividad de servicios externos.

4. **Llaves de Idempotencia (Idempotency Keys):** Para que los reintentos automáticos enviados por el API Gateway debido a desconexiones transitorias de red no creen reservas duplicadas en base de datos, el cliente debe proveer una llave única (UUID) en las cabeceras HTTP. Reservas valida la existencia de esa llave en un almacén rápido (como Redis con TTL) antes de persistir, asegurando la garantía *Exactly-Once*.

5.1.4. Pseudocódigo de Conexión Tolerante con Jitter

```
1 import time
2 import random
3
4 def conectar_con_backoff_y_jitter(max_intentos=5, base_delay=1.0,
5     max_delay=30.0):
6     intento = 0
7     while intento < max_intentos:
8         try:
9             # Intentar conexion real a PostgreSQL
10            db = conectar_postgres()
11            print("Conexion establecida con la base de datos.")
12            return db
13        except Exception as e:
14            intento += 1
15            if intento == max_intentos:
16                raise Exception("Base de datos indisponible
17                permanentemente.") from e
18
19            # Calcular delay exponencial
20            delay = min(max_delay, base_delay * (2 ** intento))
21            # Introducir Jitter (ruido aleatorio para evitar colisiones)
22            sleep_time = random.uniform(0.5 * delay, 1.5 * delay)
23
24            print(f"Error de conectividad transitorio ({e}). Reintento {
25            intento}/{max_intentos} en {sleep_time:.2f}s...")
26            time.sleep(sleep_time)
```

Listing 10: Implementación de reintentos de conexión a PostgreSQL con Backoff y Jitter

5.2. Fallo 6: Condición de Carrera (Consistencia)

5.2.1. Explicación Teórica y Fundamentos Distribuidos

La venta concurrente de asientos limitados introduce un reto clásico de consistencia de datos en sistemas distribuidos. Si dos usuarios leen de forma concurrente el estado de disponibilidad del último asiento disponible (`asiento_10` en estado disponible = `True`) y ambos

envían la transacción de compra de forma simultánea, se genera una anomalía de lectura sucia o *Lost Update* (actualización perdida), resultando en una inconsistencia catastrófica: la venta duplicada del mismo boleto físico.

Relacionado directamente con el **Teorema CAP** (Brewer [1]), este fallo demuestra el trade-off entre la Consistencia Fuerte y la Disponibilidad. Para asegurar que la base de datos de inventario permanezca consistente (C), el sistema debe sacrificar latencia bloqueando las peticiones concurrentes para procesarlas de forma estrictamente secuencial, o bien abortar transacciones concurrentes afectando temporalmente la disponibilidad percibida (A).

5.2.2. Relación con la Literatura Académica (MLR)

Kleppmann [2] detalla en su libro sobre diseño de datos que para prevenir condiciones de carrera concurrentes a nivel de aplicación (donde no basta con usar variables sincronizadas en memoria porque existen múltiples pods balanceando carga), se debe delegar la exclusión mutua al motor de base de datos relacional mediante adecuados niveles de aislamiento transaccional SQL (ANSI/ISO). El aislamiento *Read Committed* no es suficiente porque solo previene lecturas sucias, mas no evita anomalías de actualización perdida en operaciones de lectura-modificación-escritura. Se requiere el nivel de aislamiento *Serializable* o la adquisición explícita de bloqueos sobre los registros de datos.

5.2.3. Solución Propuesta a Nivel de Producción

Para garantizar consistencia y prevenir de forma absoluta la doble reserva en producción, se proponen tres alternativas de control de concurrencia:

1. **Bloqueo Pesimista en SQL (Pessimistic Locking):** Utilizar la sintaxis `SELECT ... FOR UPDATE` al consultar la disponibilidad del asiento en base de datos. Esta instrucción bloquea físicamente la fila del asiento correspondiente en PostgreSQL en un ámbito exclusivo transaccional. Cualquier transacción concurrente que intente leer el estado de esa misma fila se detendrá en espera bloqueada hasta que la primera transacción ejecute un `COMMIT` o `ROLLBACK`.
2. **Bloqueo Optimista (Optimistic Locking):** Adecuado cuando la contención sobre los asientos es baja. Se agrega una columna `version` de tipo entero en la tabla de asientos. Al actualizar, la base de datos verifica que la versión no haya cambiado: `UPDATE asientos SET disponible = FALSE, version = version + 1 WHERE id = :id AND version = :version_leida`. Si el número de filas afectadas es cero, significa que otra petición concurrente modificó el asiento primero, haciendo que el backend aborte la transacción local e informe al cliente para que elija otro asiento.
3. **Reservas Temporales Distribuidas con Holds en Redis (TTL):** Antes de persistir en PostgreSQL, se adquiere un bloqueo distribuido y rápido sobre la llave del asiento en un almacén en caché en memoria como Redis. La operación se ejecuta de

forma atómica (SET asiento:id:lock usuario_id NX EX 600), bloqueando el asiento por 10 minutos para dar tiempo a que el usuario complete el pago. Una vez pagado, se consolida la transacción en la base de datos relacional y se libera la llave en Redis.

5.2.4. Pseudocódigo de Reserva Atómica con Bloqueo Pesimista

```
1 def procesar_compra_asiento_consistente(db_connection, asiento_id,
2     usuario_id):
3     try:
4         # Iniciar transaccion explicita
5         cursor = db_connection.cursor()
6
7         # 1. Adquirir bloqueo exclusivo sobre la fila del asiento (FOR
8         UPDATE)
9         # Esto bloquea transacciones concurrentes sobre este ID de asiento
10        especifico
11        cursor.execute(
12            "SELECT disponible FROM asientos WHERE id = %s FOR UPDATE",
13            (asiento_id,)
14        )
15        row = cursor.fetchone()
16
17        if not row:
18            db_connection.rollback()
19            return {"status": "error", "message": "El asiento solicitado
20            no existe."}
21
22        disponible = row[0]
23
24        if disponible:
25            # 2. Si esta disponible, cambiar el estado a ocupado de forma
26            inmediata
27            cursor.execute(
28                "UPDATE asientos SET disponible = FALSE WHERE id = %s",
29                (asiento_id,)
30            )
31            # 3. Crear el registro en la tabla de reservas
32            cursor.execute(
33                "INSERT INTO reservas (id, asiento_id, usuario_id, estado)
34                VALUES (%s, %s, %s, 'CONFIRMADA')",
35                (generar_uuid(), asiento_id, usuario_id)
36            )
37            # Confirmar transaccion y liberar el bloqueo exclusivo
38            db_connection.commit()
39            return {"status": "success", "message": "Reserva exitosa."}
40        else:
41            # Si ya no esta disponible, liberar el bloqueo con rollback
42            db_connection.rollback()
43            return {"status": "conflict", "message": "El asiento ya fue
44            vendido a otro usuario."}
```



```

39     except Exception as e:
40         db_connection.rollback()
41         return {"status": "error", "message": f"Error interno: {e}"}

```

Listing 11: Implementación de reserva consistente utilizando Bloqueo Pesimista SQL

6. Conclusiones

La realización práctica y análisis de resiliencia en este entorno distribuido multinodo permite consolidar las siguientes conclusiones técnicas:

1. **El fallo es la regla, no la excepción:** En arquitecturas distribuidas de microservicios, las caídas transitorias y la degradación de componentes de terceros son constantes. Asumir una red y un hardware 100% confiables es un antipatrón. Las aplicaciones deben estar diseñadas para tolerar fallos (*Design for Failure*).
2. **Los patrones de resiliencia evitan caídas en cascada:** La integración de Timeouts y el patrón Circuit Breaker impide que un fallo de latencia lenta en un stub no crítico (como pagos o notificaciones) sature las conexiones locales de Reservas, previniendo el colapso de todo el ecosistema.
3. **La consistencia distribuida requiere coordinación explícita:** Tal como se analizó en la condición de carrera, para resolver el problema de la doble reserva en alta concurrencia detrás de múltiples réplicas físicas, no es suficiente el uso de memoria local; se debe delegar la exclusión mutua a mecanismos robustos como el bloqueo exclusivo de base de datos relacional o cachés distribuidas atómicas como Redis.

Referencias

- [1] Brewer, E. A. (2012). *CAP twelve years later: How the rules have changed*. IEEE Computer, 45(2), 23-29.
- [2] Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, Inc.
- [3] Vogels, W. (2009). *Eventually Consistent*. Communications of the ACM, 52(1), 40-44.
- [4] Lamport, L. (1978). *Time, clocks, and the ordering of events in a distributed system*. Communications of the ACM, 21(7), 558-565.